Lecture-19 [09 October 2026]

Simulation in Infinite Horizon Discounted Reward Models

Recall: For any $\phi : \mathcal{S} \to \Delta(\mathcal{A})$ (where $\Delta(\mathcal{A}) \to \{\phi, \phi, \dots\}$) and $\lambda \in [0,1)$,

$$V^{\phi}_{\infty,\lambda}(s) = \mathbb{E}^{\pi}\left[\sum_{n=1}^{\infty} \lambda^{n-1} r(S_{n-1}, A_{n-1}, S_n) \mid S_0 = s\right], \quad s \in \mathcal{S}.$$

Monte-Carlo Methods

* The infinite summation in the expression for $V^{\phi}_{\infty,\lambda}$ cannot be evaluated using Monte-Carlo techniques.

* Monte-Carlo techniques are based on:

a) Truncating the upper limit of the summation to some $M < +\infty$.

OR

b) Writing $V^{\phi}_{\infty,\lambda}(s) = \mathbb{E}^{\pi}\left[\sum_{n=1}^{\tau} r(S_{n-1}, A_{n-1}, S_n) \mid S_0 = s\right]$

where $\tau \sim \text{Geo}(1-\lambda)$ is independent of $\{(S_n, A_n)\}_{n \geq 0}$.

a) Truncation:

* Given a pre-specified tolerance $\varepsilon > 0$, choose M so that

$$\frac{\lambda^M}{1-\lambda} \max_{(s,a,s')} |r(s,a,s')| \leq \varepsilon.$$

Then,

$$\mathbb{E}^{\pi}\left[\sum_{n=M+1}^{\infty} \lambda^{n-1} r(S_{n-1}, A_{n-1}, S_n)\right] \leq \varepsilon,$$

and therefore

$$V^{\phi}_{\infty,\lambda}(s) \leq \underbrace{\mathbb{E}^{\pi}\left[\sum_{n=1}^{M} \lambda^{n-1} r(S_{n-1}, A_{n-1}, S_n) \mid S_0 = s\right]}_{\text{this will be evaluated using Monte-Carlo techniques}} + \varepsilon.$$

* The finite summation expectation on the RHS will be evaluated using Monte-Carlo techniques.

Algorithm 10.5: Monte-Carlo Simulation to evaluate $V^{\phi}_{\infty,\lambda}$ using Truncation

1. Initialise:

a) Specify $\phi : \mathcal{S} \to \Delta(\mathcal{A})$.
b) Specify the number of replicates K.
c) Specify the truncation level M.
d) Specify $\bar{s} \in \mathcal{S}$
e) Create an empty list VALUES.
f) Set $k \leftarrow 1$. Set $v(\bar{s}) \leftarrow 0$ specify $\{\tau_k\}_{k \geq 1}$

2. Replicate : While $k \leq K$:

a) $s \leftarrow \bar{s}$, $m \leftarrow 1$, $u \leftarrow 0$.
b) Generate a replicate : While $m \leq M$:

i) Sample $a \sim \phi(\cdot|s)$.
ii) Simulate (s', R)
OR
Sample $s' \sim P(\cdot|s,a)$ and set $R \leftarrow r(s,a,s')$.

iii) $u \leftarrow u + \lambda^{m-1} R$

iv) $s \leftarrow s'$
v) $m \leftarrow m+1$

c) Update :

i) Append u to VALUES $v(\bar{s}) \leftarrow v(\bar{s}) + \tau_k (u - v(\bar{s}))$
ii) $k \leftarrow k+1$.

3. Terminate : Return

$\hat{V}_{\alpha,\lambda}(s) = \text{mean}(\text{VALUES})$. Return $v(\bar{s})$

Remark :

* Instead of computing mean (VALUES), one can use stochastic approximation to compute

$$v(\bar{s}) \leftarrow v(\bar{s}) + \tau_k (u - v(\bar{s}))$$

after the kth replicate. See the red parts in the algorithm.

b) Geometric stopping time

* In algorithm 10.5, we first simulate $\tau \sim \text{Geo}(1-\lambda)$

* In step 2(b)(iii), we set $\lambda = 1$ so that $u \leftarrow u + R$.

Reading exercise : Go through example 10.3 in Chapter 10 of Puterman, Chan

Online TD(γ)

* Because of the infinite summation, offline TD methods face the same problems as Monte-Carlo — the summation cannot be evaluated directly.

* Hence, TD methods are usually implemented online.

Algorithm 10.6 : Online TD(γ) for estimating $v^{\phi}_{\infty,\lambda}$

1. **Initialise** :
   a) Specify $\phi : \mathcal{S} \to \Delta(\mathcal{A})$. ← eligibility trace
   b) Set $v(s) \leftarrow 0$, $e(s) \leftarrow 0$, $\text{Count}(s) \leftarrow 0$ for all $s \in \mathcal{S}$.
   c) Specify $\gamma \in [0,1]$ and learning rate sequence $\{\tau_n\}_{n \geq 1}$
   d) Specify number of iterations N
   e) Set $n \leftarrow 1$
   f) Specify $s \in \mathcal{S}$.

2. **Iterate** : While $n \leq N$:
   a) $e(s) \leftarrow e(s) + 1$ — accumulation trace
   b) $\text{Count}(s) \leftarrow \text{Count}(s) + 1$
   c) Sample $a \sim \phi(\cdot|s)$.
   d) Simulate (s', R)
      OR
      Sample $s' \sim P(\cdot|s,a)$ and set $R \leftarrow r(s,a,s')$.
   e) Evaluate the temporal difference
   $$\delta \leftarrow R + \lambda v(s') - v(s).$$
   f) For all $\bar{s} \in \mathcal{S}$, set
   $$v(\bar{s}) \leftarrow v(\bar{s}) + \tau_{\text{Count}(\bar{s})}\, \delta\, e(\bar{s}),$$
   $$e(\bar{s}) \leftarrow \lambda \gamma e(\bar{s}).$$
   g) $s \leftarrow s'$, $n \leftarrow n+1$.

3. **Terminate** : Return $v(\bar{s})$ for all $\bar{s} \in \mathcal{S}$.

Remarks :

* The discount factor λ appears in $e(s)$ update (step 2(f)) and TD evaluation (step 2(e)) steps.

* When using replacement trace, step 2(a) becomes $e(s) \leftarrow 1$.

To understand the impact of the choice of eligibility trace on the final estimate $v(s)$, see the example following Algorithm 10.6 in Chapter 10 of Puterman & Chan.

* If a state $s \in \mathcal{S}$ is visited infrequently, the corresponding $v(s)$ will not be accurate. One way to circumvent this issue is to replace $s \leftarrow s'$ in step 2(g) with

Sample $s \sim$ a pre-specified distribution on $\mathcal{S}$.

Such a process is called "random restart".

* Choosing a good initialisation (instead of $v(s) \leftarrow 0 \ \forall s \in \mathcal{S}$) may be important in some applications. One may try

$\forall s \in \mathcal{S}$: $v(s) \leftarrow (1-\lambda)\, r_\phi^{(0)}(s)$, where

$$r_\phi^{(0)}(s) := \mathbb{E}^\pi\left[r(S_0, A_0, S_1) \mid S_0 = s \right] = \sum_{a \in \mathcal{A}} r(s,a,s') \cdot \phi(a|s) \cdot P(s'|s,a)$$

can be evaluated separately (using, say, a Monte-Carlo approach).

Q-Learning and SARSA : Policy Optimisation

* We will now study simulation techniques that help us find good policies alongside value function estimation

* Recall that value iteration tries to determine the solution to

$$v(s) = \max_{a \in \mathcal{A}} \left[r(s,a) + \lambda \sum_{s' \in \mathcal{S}} P(s'|s,a)\, v(s') \right], \quad s \in \mathcal{S}.$$

Theory tells us that $\underline{v} = \underline{v}^*_{\infty,\lambda}$ is the unique solution to the above equation.

* Choosing an action according to

$$\arg\max_{a \in \mathcal{A}} \left[r(s,a) + \lambda \sum_{s' \in \mathcal{S}} P(s'|s,a)\, v(s') \right]$$

is referred to as greedy sampling.

* Define the function $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ as

$$Q(s,a) := r(s,a) + \lambda \sum_{s' \in \mathcal{S}} P(s'|s,a)\, v(s'), \qquad (s,a) \in \mathcal{S} \times \mathcal{A}.$$

$Q(\cdot,\cdot)$ is referred to as the state-action value function.

* Noting that $V^*_{\infty,\lambda}$ satisfies the Bellman equation

$$V^*_{\infty,\lambda}(s) = \max_{a\in\mathcal{A}} \left[r(s,a) + \lambda \sum_{s'\in\mathcal{S}} P(s'|s,a)\, V^*_{\infty,\lambda}(s') \right], \quad s\in\mathcal{S},$$

and defining $Q^*_{\infty,\lambda} : \mathcal{S}\times\mathcal{A} \to \mathbb{R}$ as

$$Q^*_{\infty,\lambda}(s,a) := r(s,a) + \lambda \sum_{s'\in\mathcal{S}} P(s'|s,a)\, V^*_{\infty,\lambda}(s'), \quad (s,a)\in\mathcal{S}\times\mathcal{A},$$

it follows that

$$Q^*_{\infty,\lambda}(s,a) = r(s,a) + \lambda \sum_{s'\in\mathcal{S}} P(s'|s,a) \max_{a'\in\mathcal{A}} Q^*_{\infty,\lambda}(s',a') \quad \forall (s,a)\in\mathcal{S}\times\mathcal{A},$$

$$V^*_{\infty,\lambda}(s) = \max_{a\in\mathcal{A}} Q^*_{\infty,\lambda}(s,a) \quad \forall s\in\mathcal{S}.$$

* The optimal state-action value function $Q^*_{\infty,\lambda}$ is the unique solution to the fixed-point equation

$$Q(s,a) = r(s,a) + \lambda \sum_{s'\in\mathcal{S}} P(s'|s,a) \max_{a'\in\mathcal{A}} Q(s',a').$$

(For a more detailed treatment of the theory behind Q-functions, refer to the nice tutorial (Tutorial 05) covered by the TAs).

* It is standard approach in RL to use Q-functions to determine optimal policies.

a) Action selection : Exploration vs Exploitation

* To find optimal policies, all (s,a) pairs must be visited many times so that the corresponding $Q(s,a)$ values are accurate.
* Thus, exploring new (s,a) pairs during the algorithm becomes important. This is the idea behind exploration.
* Exploitation refers to the process of picking actions greedily

$$\bar{a} \in \operatorname*{argmax}_a Q(s,a).$$

* Both exploration & exploitation must be balanced well. A typical approach when finding an optimal policy is to :

  - Begin by exploring broadly and frequently, then
  - Narrow exploration by searching among actions with large $Q(s,a)$ values
  - Finally implement greedy actions frequently.

i) **ε-greedy sampling:**

In this approach, we pick a' in state $s \in \mathcal{S}$ according to

Approach 1

$$a' = \begin{cases} a^* \in \underset{a \in \mathcal{A}}{\operatorname{argmax}}\ Q(s,a), & \text{w.p. } 1-\varepsilon, \\ \text{(ties resolved uniformly at random)} & \\ a \in \mathcal{A} \setminus \{a^*\}, & \text{w.p. } \dfrac{\varepsilon}{|\mathcal{A}|-1}. \end{cases}$$

Alternatively, one can pick

Approach 2

$$a' = \begin{cases} a^* \in \underset{a \in \mathcal{A}}{\operatorname{argmax}}\ Q(s,a), & \text{w.p. } 1-\varepsilon+\dfrac{\varepsilon}{|\mathcal{A}|}, \\ \text{(ties resolved uniformly at random)} & \\ a \in \mathcal{A} \setminus \{a^*\}, & \text{w.p. } \dfrac{\varepsilon}{|\mathcal{A}|}. \end{cases}$$

> **Exercise:** Write a closed-form expression for $\mathbb{P}(A' = a')$ when
> - $A' \sim$ approach 1.
> - $A' \sim$ approach 2.

ii) **Softmax (Boltzmann) sampling:**

In this approach, we pick action a' in state $s \in \mathcal{S}$ with probability

$$\frac{e^{\eta Q(s,a')}}{\sum_{a \in \mathcal{A}} e^{\eta Q(s,a)}},$$

where η^{-1} is referred to as inverse temperature parameter

Remark: The parameters ε and η can be tuned to have early exploration and late exploitation. This means

$$\varepsilon_n \to 0 \text{ as } n \to \infty, \qquad \eta_n \to \infty \text{ as } n \to \infty.$$

Q-Learning and SARSA Algorithms

* Start with an initial choice for $Q = [Q(s,a) : (s,a) \in \mathcal{S} \times \mathcal{A}]$.
* **Q-learning:** Observe $(s,a) \to (s', r)$, and update Q according to

$$Q(s,a) \leftarrow Q(s,a) + \tau \left(r(s,a,s') + \lambda \max_b Q(s',b) - Q(s,a) \right).$$

State Action Reward State Action

* SARSA: Observe $(s,a) \to (s', r)$, and update Q according to

$$Q(s,a) \leftarrow Q(s,a) + \tau\left(r(s,a,s') + \lambda Q(s',a') - Q(s,a)\right),$$

where a' is selected through ε-greedy or softmax sampling in state s' (whatever sampling method was used to select a in state s).

$$(s - a - r - s' - a')$$

Remark: Technically, SARSA should be termed SASRA $(s - a - s' - r - a')$, as $r = r(s,a,s')$.

a) Q-learning in Episodic Models:

Algorithm 10.9: Q-learning for an Episodic Model

1. Initialise:
   a) Set $Q(s,a) \leftarrow 0$, $\text{count}(s,a) \leftarrow 0$ for all $(s,a) \in \mathcal{S} \times \mathcal{A}$.
   b) Specify the learning rate sequence $\{\tau_n\}_{n \geq 1}$
   c) Specify $\bar{s} \in \mathcal{S} \setminus \Delta$ and $\bar{a} \in \mathcal{A}$.
   d) Specify the number of episodes K.
   e) Specify $\{\varepsilon_n\}_{n \geq 1}$ (for ε-greedy sampling) or $\{\eta_n\}_{n \geq 1}$ (for softmax sampling).
   f) $k \leftarrow 1$.

2. Iterate: While $k \leq K$:
   a) $s \leftarrow \bar{s}$, $a \leftarrow \bar{a}$.
   b) Generate an episode: While $s \notin \Delta$:
      i) $\text{count}(s,a) \leftarrow \text{count}(s,a) + 1$.
      ii) Simulate (s', R)
      OR
      Sample $s' \sim P(\cdot \mid s,a)$ and set $R \leftarrow r(s,a,s')$.
      iii) Update

$$Q(s,a) \leftarrow Q(s,a) + \tau_{\text{count}(s,a)}\left(R + \max_b Q(s',b) - Q(s,a)\right).$$

      iv) Choose $a' \in \mathcal{A}$ according to ε-greedy or softmax sampling rule by choosing

$$\varepsilon = \varepsilon_{\sum_b \text{count}(s',b)} \quad \text{or} \quad \eta = \eta_{\sum_b \text{count}(s',b)}$$

      v) $s \leftarrow s'$, $a \leftarrow a'$.

c) $k \leftarrow k+1$.

3. Terminate: Return $Q(s,a)$ for all $(s,a) \in \mathcal{S} \times \mathcal{A}$.

Remarks:

* The sampling technique used to pick actions in step 2(b)(iv) above may be represented by a decision rule $\phi : \mathcal{S} \to \Delta(\mathcal{A})$. The policy $\pi = \{\phi, \phi, \ldots\}$ is then referred to as the behavioural policy or learning policy.

$\phi \to \varepsilon$-greedy or softmax

* The Q-learning update equation features $a \sim \phi$ and $b \sim$ greedy in it. For this reason, Q-learning is also called off-policy algorithm.

* In 2(b)(iii), the $Q(s,a)$ values for all previously visited (s,a) pairs may be updated as in TD(γ), if needed.

* In every iteration, the same $(\bar{s}, \bar{a})$ need not be used. This can be changed in every iteration.

b) SARSA in Episodic Models:

Algorithm 10.10: SARSA in Episodic Models

1. Initialise:
   a) Set $Q(s,a) \leftarrow 0$, $\text{count}(s,a) \leftarrow 0$ for all $(s,a) \in \mathcal{S} \times \mathcal{A}$.
   b) Specify the learning rate sequence $\{\tau_n\}_{n \geq 1}$
   c) Specify $\bar{s} \in \mathcal{S} \setminus \Delta$ and $\bar{a} \in \mathcal{A}$.
   d) Specify the number of episodes K.
   e) Specify $\{\varepsilon_n\}_{n \geq 1}$ (for ε-greedy sampling) or $\{\eta_n\}_{n \geq 1}$ (for softmax sampling).
   f) $k \leftarrow 1$.

2. Iterate: While $k \leq K$:

   a) $s \leftarrow \bar{s}$, $a \leftarrow \bar{a}$.
   b) Generate an episode: While $s \notin \Delta$:

      i) $\text{Count}(s,a) \leftarrow \text{count}(s,a) + 1$.
      ii) Simulate (s', R)
          OR
          Sample $s' \sim P(\cdot|s,a)$ and set $R \leftarrow r(s,a,s')$.

      iii) Choose $a' \in \mathcal{A}$ using ε-greedy or softmax sampling rule.

iv) Update

$$Q(s,a) \leftarrow Q(s,a) + \tau_{\text{count}(s,a)} \left(R + Q(s',a') - Q(s,a) \right)$$

v) $s \leftarrow s'$, $a \leftarrow a'$.

c) $k \leftarrow k+1$.

3. Terminate : Return $Q(s,a)$ for all $(s,a) \in \mathcal{S} \times \mathcal{A}$.

Remarks :

* In SARSA, the same ε-greedy or softmax sampling rule is used to pick a in state s and a' in state s'! Thus, SARSA is referred to as an on-policy algorithm.
  → except maybe the first time the episode generation loop is entered

* If $\phi : \mathcal{S} \to \Delta(\mathcal{A})$ represents the sampling rule used at every step to pick actions in Q-learning and SARSA, then the successive approximation Q-iterates converge as follows :

  - Under Q-learning :

$$Q_n(s,a) \xrightarrow[a.s.]{n \to \infty} \mathbb{E}^{\phi^*} \left[r(S, A, S') + \max_{a'} Q^*_\infty(S', a') \,\middle|\, S = s, A = a \right]$$

$$= Q^*_\infty(s,a).$$

Here,

$$\phi^*(s) \in \arg\max_{a'} Q^*_\infty(s,a) \qquad \forall s \in \mathcal{S}.$$

  - Under SARSA,

$$Q_n(s,a) \xrightarrow[a.s.]{n \to \infty} \mathbb{E}^{\phi} \left[r(S,A,S') + Q^{\phi}_\infty(S',a) \,\middle|\, S = s, A = a \right]$$

$$= Q^{\phi}_\infty(s,a).$$

→ may not be optimal

* To make sure SARSA Q-iterates converge to Q^*_∞, the sampling rule must be switched to greedy after some time.

Reading exercise : Go through Example 10.6 & 10.7 in Chapter 10 of Puterman & Chan.

c) Q-learning in discounted reward models :

Algorithm 10.11 : Q-learning for a discounted model

1. **Initialise** :
   a) Set $Q(s,a) \leftarrow 0$ and $\text{count}(s,a) \leftarrow 0$ for all $(s,a) \in \mathcal{S} \times \mathcal{A}$.
   b) Specify learning rate sequence $\{\tau_n\}_{n \geq 1}$.
   c) Specify number of iterations N.
   d) Specify $\{\varepsilon_n\}_{n \geq 1}$ (for ε-greedy sampling) OR $\{\eta_n\}_{n \geq 1}$ (for softmax sampling).
   e) Specify $s \in \mathcal{S}$ and $a \in \mathcal{A}$.
   f) $n \leftarrow 1$.

2. **Iterate** : While $n \leq N$:
   a) $\text{count}(s,a) \leftarrow \text{count}(s,a) + 1$.
   b) Simulate (s', R)
      OR
      Sample $s' \sim P(\cdot | s,a)$ and set $R \leftarrow r(s,a,s')$.
   c) Update

$$Q(s,a) \leftarrow Q(s,a) + \tau_{\text{count}(s,a)} \left[R + \lambda \max_b Q(s',b) - Q(s,a) \right].$$

   d) Choose $a' \in \mathcal{A}$ according to ε-greedy rule or softmax rule by setting

$$\varepsilon \leftarrow \varepsilon_{\sum_b \text{count}(s',b)} \quad \boxed{\text{OR}} \quad \eta \leftarrow \eta_{\sum_b \text{count}(s',b)}$$

   e) $s \leftarrow s'$, $a \leftarrow a'$.
   f) $n \leftarrow n+1$.

3. **Terminate** : Return $Q(s,a)$ for all $(s,a) \in \mathcal{S} \times \mathcal{A}$.

Remarks: SARSA in discounted models replaces steps 2(c) and 2(d) above with

c) Sample $a' \in \mathcal{A}$ according to ε-greedy or softmax sampling by using

$$\varepsilon \leftarrow \varepsilon_{\sum_b \text{count}(s',b)} \quad \boxed{\text{OR}} \quad \eta \leftarrow \eta_{\sum_b \text{count}(s',b)}$$

d) Update

$$Q(s,a) \leftarrow Q(s,a) + \tau_{\text{count}(s,a)} \left[R + \lambda Q(s',a') - Q(s,a) \right].$$

* Q-learning algorithm returns a state-action value function from which we may extract the decision rule $\phi : \mathcal{S} \to \mathcal{A}$ as

$$\phi(s) \in \arg\max_{a \in \mathcal{A}} Q(s,a) \quad , \quad s \in \mathcal{S}.$$

The value of the policy $\pi = \{\phi, \phi, \cdots\}$ may then be approximated by

$$V(s) = Q(s, \phi(s)) \quad , \quad s \in \mathcal{S}.$$

Reading exercise : Go through the example immediately after Algorithm 10.11 in Chapter 10 of Puterman and Chan.